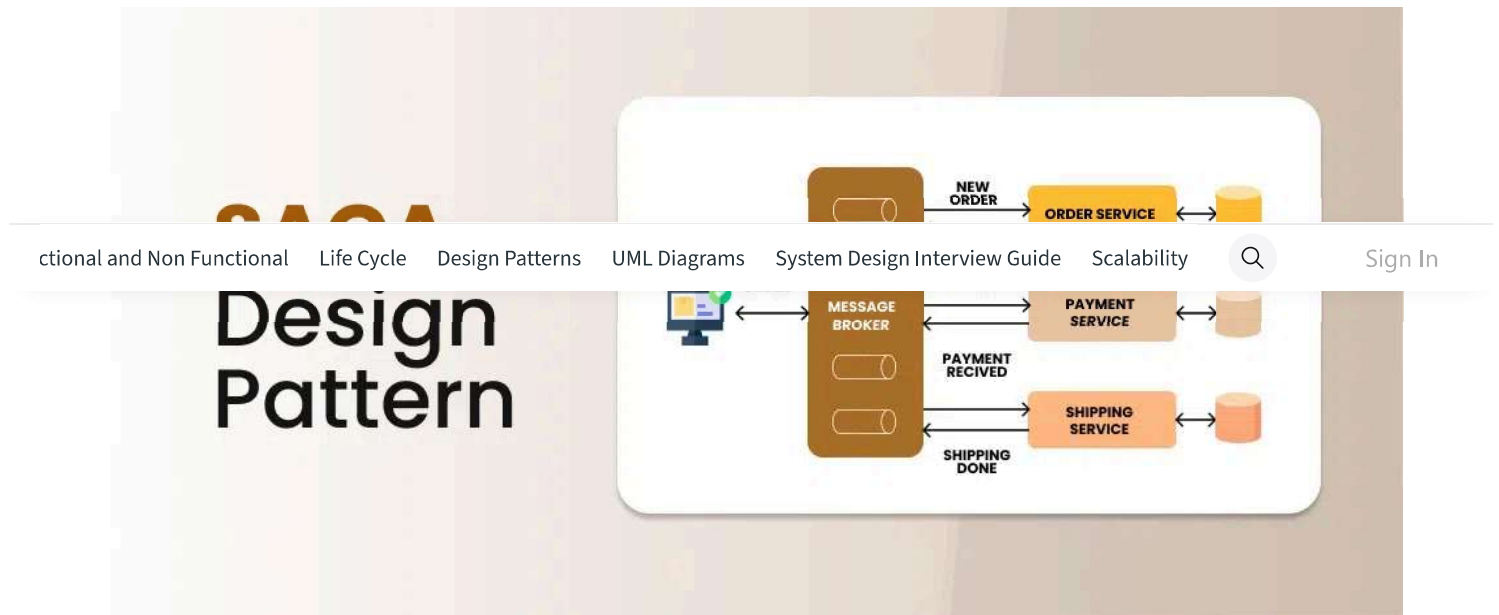


SAGA Design Pattern

Last Updated : 23 Jul, 2025

The SAGA Design Pattern is a pattern used to manage long-running and distributed transactions, particularly in [microservices architecture](#). Unlike traditional monolithic transactions, which require a single, centralized transaction management system, the SAGA pattern breaks down a complex transaction into a series of smaller, isolated operations, each handled by a different service.



SAGA Design Pattern

Table of Content

- [What is a Distributed Transaction?](#)
- [What is 2PC for Distributed Transaction Management?](#)
- [Problems with Traditional Distributed Transaction Protocols](#)
- [What is the SAGA Design Pattern?](#)
- [Why do we need SAGA Design Pattern?](#)
- [How SAGA Design Pattern Works?](#)
- [Example of SAGA Design Pattern](#)
- [Approaches to Implement SAGA Design Pattern](#)
- [Advantages and Disadvantages of SAGA Pattern](#)

What is a Distributed Transaction?

A distributed transaction refers to a type of transaction that involves multiple, separate systems or databases, often spread across different locations or networks, which need to work together to complete a task. It's like a coordinated team effort, where each system handles a small part of the work, but they all must complete their respective tasks successfully for the overall transaction to be considered successful.

For example, imagine you're making an online purchase. The transaction might involve:

- Your bank checks if you have enough funds.
- The e-commerce platform reserving the product you're buying.
- A shipping service getting ready to send your item.
- A warehouse updating its stock levels

What is 2PC for Distributed Transaction Management?

2PC (Two-Phase Commit) is a protocol used to ensure all participants in a distributed transaction either commit or abort, ensuring consistency. In the first phase, the coordinator asks all participants to agree to commit, and in the second phase, participants either vote to commit or abort the transaction.

Problems with Traditional Distributed Transaction Protocols

Traditional distributed transaction protocols like Two-Phase Commit (2PC) have limitations in modern systems, primarily due to:

- **Blocking Nature:** If the coordinator fails after initiating the transaction, participants may be left waiting indefinitely, causing delays.
- **Single Points of Failure:** The coordinator is crucial for decision-making. If it crashes, the entire transaction can get stuck, impacting reliability.
- **Network Partitions:** If the network splits, some nodes might not receive the final decision, leading to inconsistent states (i.e., some nodes might commit while others don't), which causes data inconsistency.

These problems make 2PC unsuitable for modern, highly available, and fault-tolerant systems.

What is the SAGA Design Pattern?

The SAGA Design Pattern is a pattern used to manage long-running, distributed transactions in a microservices architecture.

- Instead of relying on traditional, monolithic transactions (which require locking databases across services), the SAGA pattern breaks the transaction into a series of smaller, independent operations that each belong to a different service.
- These smaller operations, also called saga steps, are executed sequentially or in parallel, with each step compensating for potential failures in the others.

Why do we need SAGA Design Pattern?

SAGA is needed because 2PC, while simple, doesn't work well in distributed systems where availability and fault tolerance are critical. In real-world systems, network failures, crash recovery, and long-running transactions are common, and 2PC's blocking behavior and reliance on a single coordinator can make the system unreliable and slow. **SAGA** provides a more flexible, decentralized approach to managing long-running distributed transactions.

***SAGA** addresses the limitations of **2PC** by breaking a transaction into smaller, independent steps, each with its own compensating action if something goes wrong. Here's how it solves*

key issues of 2PC:

- **No Blocking:** Unlike 2PC, which blocks participants until the transaction is complete, SAGA allows each step to execute independently, avoiding delays due to coordinator failure.
- **No Single Point of Failure:** SAGA doesn't rely on a central coordinator. Each step is independent, so if one step fails, it doesn't block the entire process.
- **Graceful Failure Handling:** Instead of rolling back the entire transaction (as in 2PC), SAGA uses compensating actions to undo successful steps if a failure occurs, ensuring the system remains consistent.
- **Resilience to Network Partitions:** SAGA can continue even if parts of the network fail, because each step is independent, and compensating actions can be executed later when the partition resolves.

How SAGA Design Pattern Works?

The SAGA Design Pattern manages long-running distributed transactions by breaking them into smaller steps, each with its own compensating action in case of failure. Here's how it works:

- **Breaking Down the Transaction:** A big transaction is divided into smaller, independent sub-transactions (steps), each handled by different services. For example, reserving a product, charging the customer, and shipping the item.
- **Independent Execution:** Each step runs independently without waiting for the others to finish. If a step succeeds, the next step proceeds.
- **SAGA Execution Coordinator:** The SAGA Execution Coordinator manages and coordinate the flow of these steps, triggering each one in sequence.
- **Compensating Actions:** If any step fails, the system doesn't roll back everything. Instead, it executes compensating actions to undo the work done in previous successful steps, like refunding a payment or canceling a reservation.
- **SAGA Log:** Helps manage and track the state of a long-running distributed transaction, ensuring that all steps are completed successfully or properly compensated in case of failure.

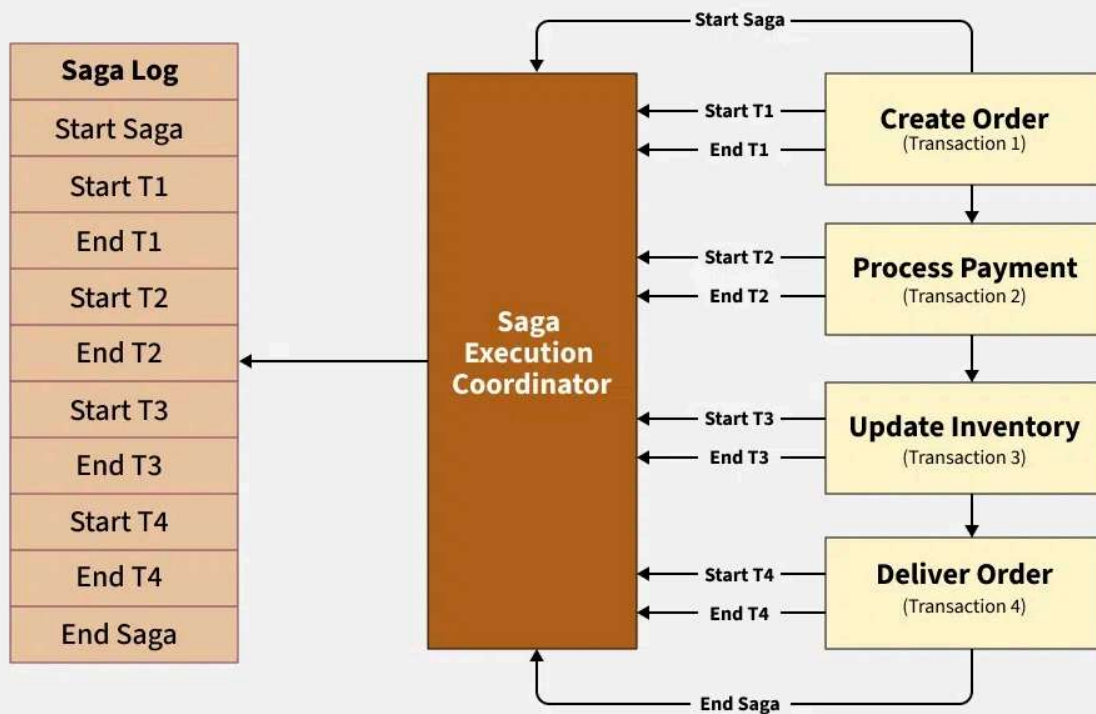
Example of SAGA Design Pattern

Let's understand how SAGA works using the example of an e-commerce order process with the SAGA Execution Coordinator and SAGA Log.

- **Step 1: Create Order:** Reserve the product.
- **Step 2: Process Payment:** Charge the customer's card.
- **Step 3: Update Inventory:** Reduce the stock.
- **Step 4: Deliver Order:** Ship the product to the customer.

How It Works:

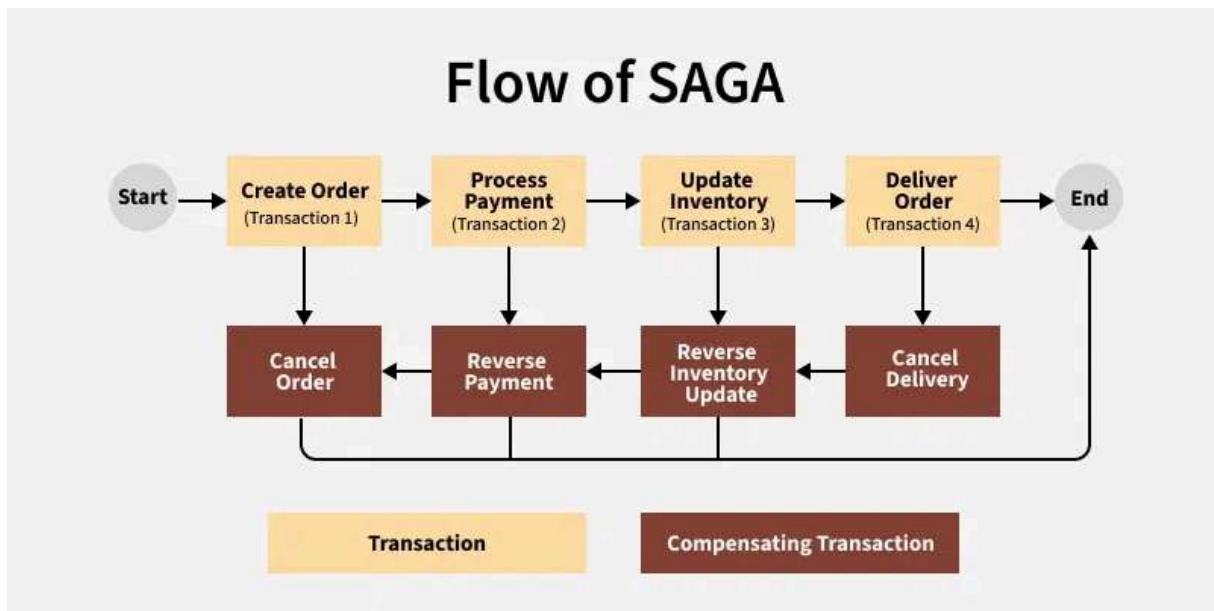
Example of SAGA Design Pattern



Example of SAGA Design Pattern

- The SAGA Execution Coordinator manages the flow of these steps, triggering each one in sequence.
- Each step has a compensating action that is triggered if something goes wrong (e.g., if payment fails, the product is unreserved).
- The SAGA Log tracks the state of each step. It logs each step as in-progress, completed, or failed, and records any compensating actions needed.

Flow of SAGA:



Flow of SAGA

- **Start the SAGA:**
 - The process begins by executing the first step in the sequence.
- **Execute Step 1:**
 - The system performs the first sub-transaction (e.g., creating the order and reserving the product). If this step is successful, move to Step 2. If it fails, trigger its compensating action (e.g., cancel the order) and stop.
- **Execute Step 2:**
 - If Step 1 was successful, the next step (e.g., process the payment) is executed. If Step 2 fails (e.g., payment is declined), its compensating action (e.g., refund the payment) is triggered, and Step 1's compensating action (e.g., unreserve the product) is also executed.
- **Execute Step 3:**
 - If Step 2 was successful, proceed to the next step (e.g., update inventory). If Step 3 fails, its compensating action (e.g., reverse inventory update) is triggered, and Step 2's compensating action (e.g., refund payment) is executed.
- **Execute Step 4:**
 - Finally, if all previous steps are successful, the last step (e.g., deliver the order) is executed. If any prior step has failed, its compensating actions are triggered, ensuring the system remains consistent.

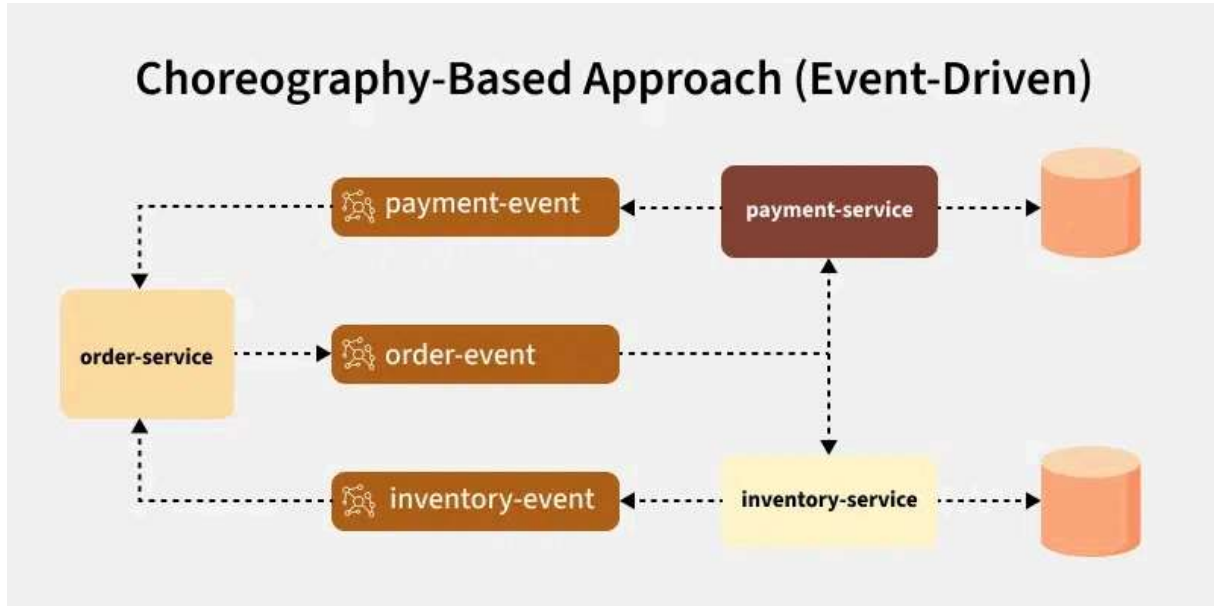
Approaches to Implement SAGA Design Pattern

Below are the two main approaches to implementing the SAGA pattern:

1. Choreography-Based Approach (Event-Driven)

- There is no central coordinator; each service knows what to do next and triggers the next step by emitting events.

- Services communicate through events (e.g., via message queues or event streams). Each service listens for events and performs its own action when the event occurs.
- Services operate independently and don't need to know about each other's details, just the events they need to react to.
- If a service fails, it publishes a failure event, and other services can listen to it and perform compensating actions.



Choreography-Based Approach (Event-Driven)

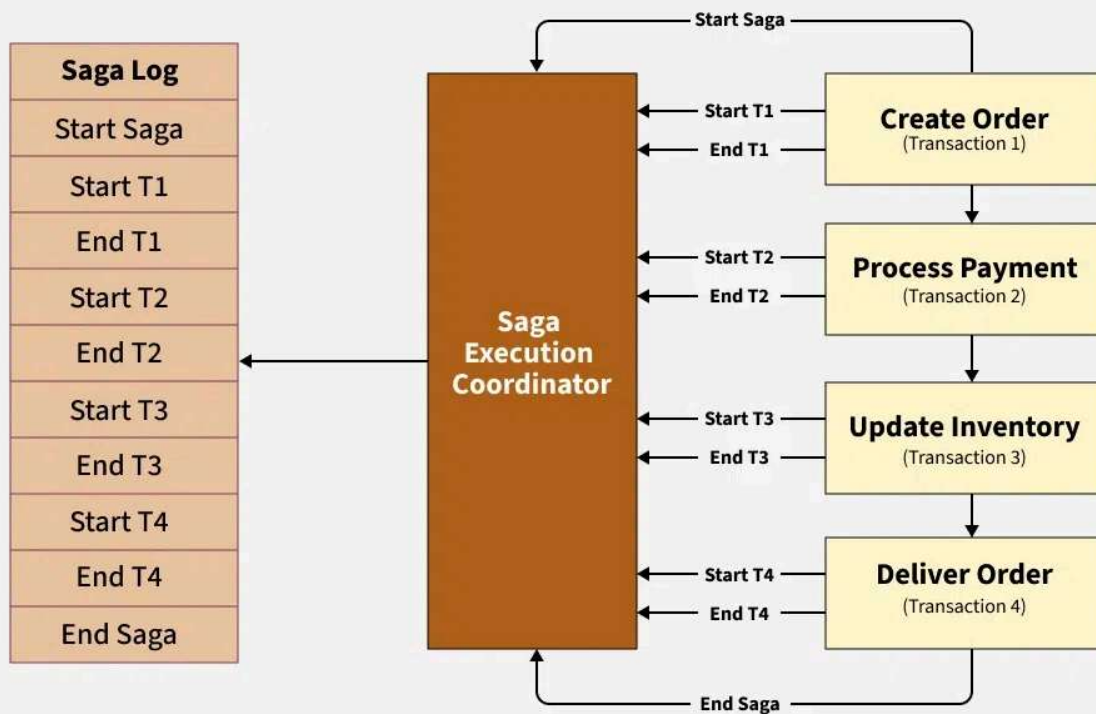
Example:

- **Order Service** creates an order and publishes the "OrderCreated" event.
- **Payment Service** listens for "OrderCreated" and processes payment, publishing "PaymentProcessed".
- **Inventory Service** listens for "PaymentProcessed" and updates inventory, and so on.

2. Orchestration-Based Approach (Centralized)

- A single SAGA Execution Coordinator (orchestrator) controls the flow of the entire saga.
- The orchestrator tells each service when to start, what to do, and when to proceed to the next step.
- The orchestrator has detailed knowledge of each service and their responsibilities in the saga.
- The orchestrator manages failure recovery and calls the compensating actions if needed.

Orchestration-Based Approach (Centralized)



Orchestration-Based Approach (Centralized)

Example:

- The SAGA Execution Coordinator starts the saga and tells the Order Service to create the order.
- Once the Order Service succeeds, the orchestrator tells the Payment Service to process the payment, and so on.

Advantages and Disadvantages of SAGA Pattern

Below are the main advantages and disadvantages of SAGA Pattern:

Advantages of SAGA Pattern	Disadvantages of SAGA Pattern
With SAGA, if one step fails, the entire process can be rolled back or compensated without affecting other steps.	Implementing SAGA requires additional coding and architecture to handle compensation and rollback steps.
SAGA provides a clear and standardized way to handle errors and compensations, making it easier to debug and maintain.	Not all frameworks or platforms support SAGA out of the box, which can make implementation more difficult.
SAGA can support asynchronous processing, allowing for greater concurrency and	The SAGA pattern requires careful design to ensure that the compensations and rollbacks are implemented

performance.	correctly and can handle all possible failure scenarios.
SAGA can handle transactions across multiple services or databases, allowing for more scalable and distributed architectures.	SAGA may result in additional latency due to the need to coordinate between different services or databases.

Overall, the SAGA pattern is a powerful tool for managing distributed transactions and providing fault tolerance. However, it requires careful design and implementation to ensure that it is used effectively and does not introduce additional complexity or latency.

Comment

A

abhish...

+ Follow

12

Article Tags: [Design Pattern](#) [System Design](#)



Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305



- Company

About Us

Legal

Privacy

Policy

Careers

Contact Us

Corporate

Solution

Campus

Training
- Explore

POTD

Practice

Problems

Connect

Blogs

Upskill

Courses
- Tutorials

Programming

Languages

DSA

Web

Technology

AI, ML &

Data Science

DevOps

CS Core

Subjects

GATE

School

Subjects

Software and

Tools
- Courses

ML and Data

Science

DSA and

Placements

Web

Development

Data Science

Programming

Languages

DevOps &

Cloud

GATE

Trending

Technologies
- Offline Centers

Noida

Bengaluru

Pune

Hyderabad

Kolkata
- Preparation Corner

Interview

Corner

Aptitude

Puzzles

GfG 160

System Design